1.Abstract Class

Data **abstraction** is the process of hiding certain details and showing only essential information to the user.

Abstraction can be achieved with

1.abstract classes

2.interfaces

- **Abstract class:**
  - is a restricted class that cannot be used to create objects
  - to access it, it must be inherited from another class
- **Abstract method:**
  - can only be used in an abstract class
  - it does not have a body.
  - The body is provided by the subclass (inherited from).

Example

An abstract class can have both abstract and regular methods:

```
abstract class Animal {
  public abstract void animalSound();
  public void sleep() {
    System.out.println("Zzz");
  }
}
```

it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // will generate an error
```

important observations about abstract classes

1. An instance of an abstract class cannot be created.

2. Constructors are allowed.

3. It can have abstract and non-abstract methods.

4. We can have an abstract class without any abstract method.

5. There can be a final method in abstract class ,but any abstract method in class(abstract class) can not be declared as final . it will yield an error: “Illegal combination of modifiers: abstract and final”

6. If a class contains at least one abstract method then compulsory should declare a class as abstract

7. If the Child class is unable to provide implementation to all abstract methods of the Parent class then we should declare that Child class as abstract so that the next level Child class should provide implementation to the remaining abstract method

2. Interfaces in Java

Why do we use an Interface?

- It is used to achieve abstraction.

- Since java does not support multiple inheritances in the case of class, by using an interface it can achieve multiple inheritances.

- Any class can extend only 1 class but any class can implement infinite number of interface.

- Interfaces can have abstract methods and variables. It cannot have a method body.

- Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body).

- An interface is implicitly abstract. You do not need to use the abstract keyword while declaring an interface.

- Each method in an interface is also implicitly abstract, so the abstract keyword is not needed.

Syntax

```
modifier interface interfaceName {
    //constant declarations
    //method signatures
}

modifer class className implements interfaceName {
   //methods from the interface
};
```

- If More than one interface separate with comma.

Java Nested Interface

An interface, i.e., declared within another interface or class, is known as a nested interface.

The nested interfaces are used to group related interfaces so that they can be easy to maintain.

The nested interface must be referred to by the outer interface or class. It can't be accessed directly.

Syntax of nested interface which is declared within the interface

```
interface interface_name{
 ...
 interface nested_interface_name{
  ...
 }
}
```

Syntax of nested interface which is declared within the class

```
class class_name{
 ...
 interface nested_interface_name{
  ...
 }
}
```

3.Packages in java

Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces.

Advantage of Java Package

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.

2) Java package provides access protection(classes inside a package that are not accessible by code outside that package)

3) Java package removes naming collision.(For example there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee

Types of packages
Two types of packages

- Built-in Packages
- User-defined Packages

1. **Built-in Packages**

    - Java.lang
    - java.io
    - java.util
    - java.net

To use a class or a package from the library, you need to use the import keyword:

```
import package.name.Class;   // Import a single class

import package.name.*;   // Import the whole package
```

example

- java.io.InputStream
- Java.io.*

2. User-defined Packages

The user-defined packages are the packages created by the user. User is free to create their own packages.

We use the package keyword to create or define a package in java programming language.

Syntax

Package packagename;

The package statement must be the first statement in the program.

```
package myPackage;

public class DefiningPackage {

        public static void main(String[] args) {

                System.out.println("This class belongs to myPackage.");

        }

}
```

When we use IDE like Eclipse, Netbeans, etc. the package structure is created automatically.

Finding packages in java

1. By default the java run-time system considers the current directory which you are working as starting directory and checks for the package defined and finds it if it is present that directory.
2. To specify the directory path by setting the CLASSPATH environment variable.
3. Using -classpath option while compiling and running the java program to specify the path to your classes.

4.Static Keyword in Java

- The most fundamental reason for the widespread usage of **static** keyword in Java is to efficiently manage our program memory.

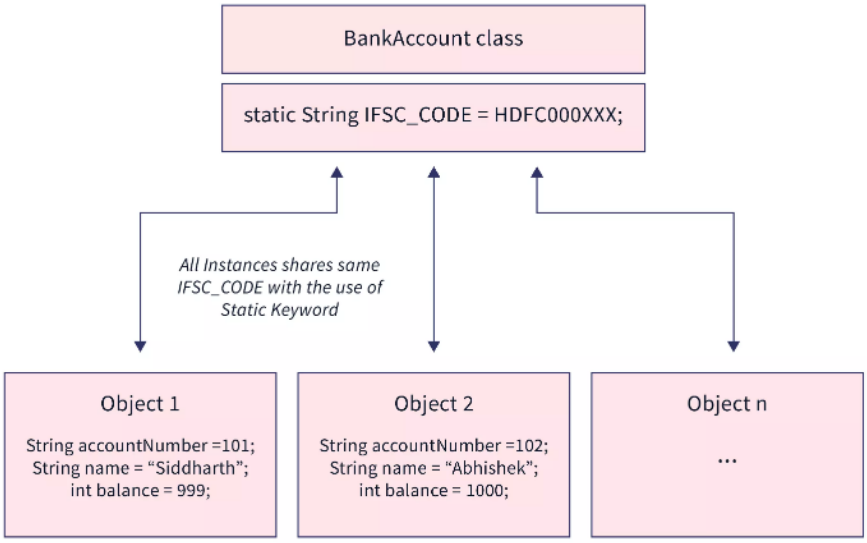
- In general, we first have to make an instance/object of a class in order to access variables or methods declared and defined inside the class.

- However, there may be times when you simply need to access a few of a class's methods or variables and don't want to create a new instance of the class merely to access these members.

- Static methods/attributes can be accessed without creating an object of a class.

- We can apply static keyword with variables, methods, blocks and nested classes.

Let's understand this with a real-life example, we know that in India a particular branch of a bank has a common IFSC_CODE for all their account holders. Here, we can make the IFSC_CODE a static member variable of the BankAccount class, so that the IFSC_CODE variable will belong to the branch of the bank (a class) with equal value for all bank account holders (all instances) rather than to a single bank account holder (a particular instance) of this class.



using static members is beneficial when you want certain properties or behaviors to be shared across all instances of a class or when you want to associate them directly with the class itself rather than with instances.